Coordinate Converter: How to Run It

Oct 29, 2026 · @Samuel

What you need

You need Python 3.9 or newer, four Python packages, and the project files from GitHub. Setup takes about 5 minutes.

Item	Why	Notes
Python 3.9+	Runs everything	Get it from python.org. On Windows, check **Add python.exe to PATH** during install.
numpy, pandas	Fast math on large files	Installed by `requirements.txt`
pyproj	ECEF to lat/lon/alt on WGS84	Installed by `requirements.txt`
astropy	Rotating space-fixed OEM frames (EME2000, GCRF, ICRF, TEME)	Installed by `requirements.txt`. Not needed for CSV or Earth-fixed OEM files.
Tkinter	The desktop window	Built into Python on Windows and macOS. Linux: `sudo apt install python3-tk`

Files in the project:

File	What it is
`xyz_to_lla.py`	The converter. Run it from the command line or import it in your own code.
`converter_gui.py`	The desktop app. Needs `xyz_to_lla.py` in the same folder.
`requirements.txt`	The package list for pip
`examples/sample.oem`	A 2-segment test orbit file (ITRF with day-of-year epochs, EME2000 with calendar epochs)

File	What it is
`examples/sample.csv`	3 test points in ECEF meters (Huntsville, North Pole, 0°/0°)

Setup (one time)

Download the code, open a terminal in its folder, and install the packages. On Windows use `py` where this guide says `python`; on Mac use `python3`.

1. **Download the code.** On your GitHub repo page, click the green **Code** button, then **Download ZIP**. Unzip it somewhere easy, like your Documents folder.
   - Or, if you have git: `git clone https://github.com/YOUR-USERNAME/coordinate-converter.git`
2. **Open a terminal in that folder.**
   - Windows: open the folder in File Explorer, click the address bar, type `cmd`, press Enter.
   - Mac: right-click the folder in Finder, then **New Terminal at Folder** (or open Terminal and type `cd` then drag the folder in).
3. **Optional: make a virtual environment.** This keeps the packages for this project separate from everything else.
   - Windows: `py -m venv .venv` then `.venv\Scripts\activate`
   - Mac/Linux: `python3 -m venv .venv` then `source .venv/bin/activate`
   - You'll see `(.venv)` at the start of the prompt. Run the activate command again each time you open a new terminal.
4. **Install the packages:** `python -m pip install -r requirements.txt`
5. **Check it worked:** `python xyz_to_lla.py examples/sample.csv test_output.csv`
   - You should see `Done. Wrote 3 points to test_output.csv`. Open that file and the Huntsville row should show about 34.7304, -86.5861, 195 m.

Run the desktop app

In the terminal from setup, run `python converter_gui.py`. A window opens. If you made a virtual environment, activate it first.

1. **Pick the input.** Click the top **Browse...** and choose a `.oem`, `.txt`, or `.csv` file. The **Save as** box fills in automatically with `<name>_lla.csv` in the same folder.
2. **Check the input type. Auto** reads `.oem` files as OEM and everything else as CSV. If your orbit file ends in `.txt`, pick **OEM**.
3. **Set the options.** Only the options for your input type are clickable.
   - OEM, **Epoch format: As in file** copies epochs exactly. **Gregorian** writes calendar dates (`2026-292T12:00:00` becomes `2026-10-19T12:00:00`). **MJD** writes Modified Julian Dates (`61332.500000000000`).
   - OEM, **Fast mode:** only matters for space-fixed frames (EME2000, J2000, GCRF, ICRF). About 8x faster, under 0.05 mm difference. Use it for big files.
   - CSV, **Column names:** type the names of your X, Y, Z columns if they aren't `x`, `y`, `z`. Names are case sensitive.
4. **Click Convert** (or press Enter). The bar animates while it runs and fills when done. The status line shows the point count.
5. **Click Open folder** to jump to the output file.

If something is wrong with the file, a popup says what and on which line. Nothing is written over your input file.

Run from the command line

The pattern is `python xyz_to_lla.py INPUT OUTPUT [options]`. Run `python xyz_to_lla.py --help` to see every option.

```
python xyz_to_lla.py examples/sample.csv out.csv
python xyz_to_lla.py examples/sample.csv out.csv --x X_m --y Y_m --z Z_m
python xyz_to_lla.py examples/sample.oem out.csv
python xyz_to_lla.py examples/sample.oem out.csv --time-format mjd
python xyz_to_lla.py examples/sample.oem out.csv --time-format gregorian --
fast
python xyz_to_lla.py orbit.txt out.csv --format oem
```

Option	What it does	Default
`--format auto|csv|oem`	Input type. Auto treats `.oem` as OEM, everything else as CSV.	`auto`

Option	What it does	Default
`--x`, `--y`, `--z`	CSV column names for X, Y, Z (meters)	`x`, `y`, `z`
`--time-format original|gregorian|mjd`	OEM only. How the epoch column is written.	`original`
`--fast`	OEM space-fixed frames only. Interpolated precession-nutation.	off
`--chunksize N`	Rows processed at a time. Lower it if you run out of memory.	1,000,000 CSV, 200,000 OEM

Paths with spaces need quotes: `python xyz_to_lla.py "my orbit.oem" "my output.csv"`.

Understanding the output

The output is always a CSV. CSV input keeps all your original columns and adds `lat`, `lon`, `alt`. OEM input produces the columns below.

Column	Meaning	Units
`object`	`OBJECT_NAME` from the META block	text
`epoch` or `epoch_mjd`	Time of the point, in the format you picked	see below
`time_system`	The file's `TIME_SYSTEM` (UTC, TAI, TT, TDB, GPS, ...)	text
`x_ecef_m`, `y_ecef_m`, `z_ecef_m`	Earth-fixed position (rotated from space-fixed if needed)	meters
`lat`, `lon`	Geodetic latitude and longitude on WGS84	degrees
`alt`	Height above the WGS84 ellipsoid (not sea level)	meters

Epoch formats. Gregorian is the normal calendar date, `YYYY-MM-DDThh:mm:ss`. MJD counts days since midnight on November 17, 1858, so noon on October 19, 2026 is `61332.5`. MJD is written with 12 decimal places, about 0.1 microsecond.

Time systems stay the same. Converting the format never shifts the time. A TDB epoch is still TDB in MJD form, so check the `time_system` column before comparing times from different files.

Troubleshooting

Problem	Fix
`'python' is not recognized` (Windows)	Use `py` instead, or reinstall Python with **Add python.exe to PATH** checked.
`ModuleNotFoundError: No module named 'pyproj'` (or pandas, numpy)	Run `python -m pip install -r requirements.txt`. If you use a virtual environment, activate it first.
`No module named 'tkinter'` (Linux)	`sudo apt install python3-tk`
`No module named 'xyz_to_lla'` when starting the app	Keep `converter_gui.py` and `xyz_to_lla.py` in the same folder, and make sure the converter is named exactly `xyz_to_lla.py`.
"needs astropy" error	Your OEM file uses a space-fixed frame. Run `python -m pip install astropy`.
Warning about IERS tables	astropy couldn't download current Earth orientation data. Results still work; connect to the internet for the most accurate rotation.
"Column(s) ... not found"	Your CSV uses different column names. The message lists the real ones; type them into the X/Y/Z boxes or use `--x --y --z`.
"does not look like a text (KVN) OEM file"	The file is XML OEM or not OEM at all. Only the text format is supported.

Problem	Fix
"is not a real date or time"	That line has an impossible epoch, like February 30 or day 366 in a non-leap year. Fix that line in the file.
Output file is open in Excel and won't save	Close it in Excel, then convert again.